



中国科学技术大学

专业学位研究生学位申请成果开题报告

学位申请成果题目: 基于 Agent 的语音识别模型自动化优化框架设计与实现

学生姓名: 沙漠

学生学号: SA24225294

校内导师: 王鹏焜

第二导师: (选填)

实践导师: 吴长印

所在院系: 软件学院

专业类别: 电子信息 0854

专业领域: 软件工程 085405

填表日期: 2026.6.24

中国科学技术大学研究生院培养处

二零二五年三月 制表

说 明

1. 学位申请成果的开题报告是保证研究生培养质量的一个重要环节，为了加强对研究生培养的过程管理，规范其学位申请成果的开题报告，特制此表。

2. 学位申请成果开题报告，应该在学位授予点或培养单位组织的学术报告会上报告，听取意见，论证后再填写此表。

3. 此表经导师签字后，交培养单位研究生教学管理办公室存档。

4. 研究生在提交学位申请时，必须提交该学位申请成果开题报告。

一、简况

研究生简况	姓 名	沙漠	学 号	SA24225294	性 别	男
	培养方式	☒ 全日制 ☐ 非全日制				
☒ 专业学位 硕士	☐ 专题研究类论文 ☐ 调研报告 ☐ 案例分析报告 ☐ 产品设计（作品创作） ☒ 方案设计 ☐ 其他 _____					
☐ 专业学位 博士	学位论文申请： ☐ 专题研究类论文 实践成果申请： ☐ 重大装备 ☐ 仪器设备 ☐ 其他硬件产品 ☐ 软件产品 ☐ 设计方案 ☐ 技术标准 ☐ 其他 _____					
学位申请 成果题目	中 文	基于 Agent 的语音识别模型自动化优化框架设计与实现				
	英 文	Design and Implementation of an Agent-Based Automated Optimization Framework for Speech Recognition Models				
学位申请成果摘要和关键词	摘 要	语音识别（ASR）是智能摄像头等物联网设备人机交互的核心入口，其“唤醒词+控制指令”范式对特定场景的识别准确率、唤醒词鲁棒性与远场适应能力要求很高。将通用预训练模型适配到具体业务、达到产品级水平，需经历数据工程、模型训练与评估分析的多轮迭代优化。该过程长期依赖零散脚本与人工经验：决策不可复现、难以传承，实验线性管理、易重复无效尝试而浪费算力，各阶段缺乏统一编排，元数据分散、评测缺乏系统化管理，难以跨实验对比。 针对上述问题，本文设计并实现一套基于 Agent 架构的 ASR 模型自动化优化平台。平台以自主设计的 LLM Agent 决策引擎为核心，按“思考—执行—反思”闭环驱动优化，并采用“通用工具层+领域技能层”双层架构——将 ASR 优化的数据准备、增强、训练、推理、评测、归因等阶段封装为统一接口的可独立调用技能，由 Agent 依据优化状态自主编排，而非固定流水线。平台融合规则型与 LLM 型评估器并将分析报告结构化持久化，构建 Trace 链路观测实现决策全程可追溯，引入配置、数据集与模型的三方版本管理保证实验可复现，并通过实验树与分层记忆沉淀经验、规避无效方向；系统以 Vue.js、FastAPI、MySQL/Redis 全栈实现，支持多用户隔离与远程多 GPU 训练适配。创新性在于将 Agent 自主决策与平台化的评测、版本和观测能力在 ASR 垂直领域深度融合。 本文预期形成一套可部署、支持团队协作的 Agent 驱动优化平台，并在 AISHELL 与真实业务数据集上完成端到端验证。预期降低对个人经验的依赖、将优化经验沉淀为可复用知识，提升迭代效率并减少算力浪费，实现实验可复现、可回溯、可审计，在唤醒词与远场等场景有效提升识别准确率，为工业级 ASR 的持续迭代提供端到端平台支				
		关键词				

		撑，并可推广至其他领域模型的自动化优化。	
	关键词	关键词数量三至五个，主题词之间空一格（英文用“ / ”分隔）	
		中文	语音识别 大语言模型智能体 自动化优化 版本管理
		英文	Speech Recognition/Large Language Model Agent/Automated Optimization/Version Management
选题来源	工程博士必须填写研究所依托的重大工程项目，并标明项目编号。 实习项目		

二、选题依据

阐述该选题的研究意义，或工程设计的价值和意义，国内外概况和发展趋势，选题的先进性和实用性，技术难度及工作量。附上主要参考文献（列出作者、论文名称、期刊名称、出版年月）。

1. 选题的研究意义

随着智能家居、车载交互与智能客服等领域的快速发展，语音识别（Automatic Speech Recognition, ASR）技术已成为人机交互的核心入口之一^[1]。特别是以萤石智能摄像头为代表的物联网（IoT）设备，普遍采用“唤醒词+控制指令”的语音交互范式。这类设备部署在家庭、办公等多样化场景中，对特定业务场景下的识别准确率、唤醒词鲁棒性及远场噪声环境的适应能力提出了极高的要求。

将通用预训练 ASR 模型适配到具体业务场景并使其达到产品级要求，本质上是一个复杂的多轮迭代优化过程。这一过程涉及数据工程（数据采集、清洗、增强）、模型训练（配置调优、分布式训练）与评估分析（多维度指标计算、错误归因）三个层面的紧密协作。在实际业务实践中，ASR 模型的优化迭代长期以零散脚本与大量人工经验判断的方式运行。算法工程师需要反复执行“分析当前模型瓶颈→提出优化假设→调整数据或配置→执行训练→解读评估结果→决定下一步方向”的循环。

这种工作模式存在多方面的突出问题。优化决策高度依赖个人经验，且不可复现也不可传承。资深工程师离职后，其积累的优化经验随之丧失，新人需要重新摸索。实验管理呈线性结构，缺乏可回溯的搜索空间管理。工程师容易重复尝试已被验证无效的优化方向，造成算力与时间的浪费。例如，“增加速度扰动增强”在某配置下已经验证无效，但三个月后另一位工程师可能因为不知情而再次尝试。各数据处理阶段以独立脚本运行，缺乏统一的编排与状态管理。一个完整的优化迭代可能涉及数据切分、增强、合并、训练清单生成、训练、评估等多个步骤，中途某步失败后往往需要从头重跑，浪费 GPU 算力。训练过程的元数据（使用的配置版本、数据来源、产物之间的依赖关系）分散存储在文件系统各处，缺乏结构化的存储和查询能力，无法方便地进行跨实验对比分析^[2]。评测流程缺乏系统化管理。测试集、评估指标与评估结果之间的关联是临时性的，评估报告以文本文件形式散落在各目录中，无法进行历史趋势分析或多实验横向对比。

近年来，基于大语言模型（Large Language Model, LLM）的智能 Agent 技术取得了突破性进展^{[3][4]}。Agent 通过“推理—行动—观察”的循环模式，结合工具调

用 (Tool Use) 与记忆机制 (Memory), 能够自主完成复杂的多步决策任务。将 Agent 架构引入 ASR 模型优化领域, 以 Agent 作为智能决策核心驱动优化闭环, 同时配合系统化的评测模块、版本管理、决策 Trace 观测与 Web 管理平台, 可以形成一套完整的智能化模型优化工程系统。这一方案既能将资深工程师的领域经验结构化地沉淀到 Agent 的记忆系统中实现知识的可复用, 又能通过平台化的管理能力使实验数据、决策过程与产物关系均可被持久化记录、查询与审计。

因此, 本课题以实际业务中 ASR 模型优化的工程需求为出发点, 设计并实现一套基于 Agent 架构的语音识别模型自动化优化平台, 旨在将 Agent 的智能决策能力与系统化的评测、版本管理、观测能力相结合, 为 ASR 模型的持续迭代优化提供端到端的平台支撑。

2. 国内外概况和发展趋势

(1) 国外研究与发展趋势

在端到端语音识别方面, 国外研究起步较早、体系较为完整。近年来, 以 Transformer 和 Conformer^[5] 为骨干网络的注意力机制模型在多个基准任务上取得了领先性能, 其中非自回归 (Non-autoregressive, NAR) 模型因其并行解码优势和低延迟特性而受到工业界广泛关注; 在数据增强方面, SpecAugment^[6] 通过对频谱进行时频掩码, 显著提升了模型对噪声和变体的鲁棒性。

在大语言模型与智能体方面, Yao 等人提出的 ReAct 范式^[3] 将推理 (Reasoning) 与行动 (Acting) 相结合, 使 Agent 能够在与外部环境交互的过程中逐步推理与决策; Shinn 等人提出的 Reflexion^[4] 通过语言形式的反馈机制使 Agent 具备从历史失败经验中学习的能力; Schick 等人的 Toolformer^[7] 证明了 LLM 能够自主判断何时以及调用哪个外部工具。在应用层面, Chan 等人提出的 MLE-bench^[8] 基于真实 Kaggle 竞赛构建了机器学习工程评测基准, 系统考察了语言智能体自主完成数据处理、模型训练与实验迭代调优的能力, 进一步表明 “Agent+树搜索+记忆” 技术路线在实验优化类任务上的有效性。Sculley 等人^[2] 早在 2015 年便指出机器学习系统中隐藏的技术债务问题, 包括数据依赖不透明、实验结果不可复现等, 推动了 MLOps 的发展。总体来看, 国外研究在端到端建模、智能体范式与工程化工具方面起步较早、积累深厚, 并正朝着工具调用更可靠、决策更可控、能力更模块化的方向演进。

(2) 国内研究现状与发展趋势

国内研究机构与企业在语音识别取得了一系列具有国际影响力的成果。高忠孝等人提出的 Paraformer 模型^[9] 通过连续积分与解触 (Continuous

Integrate-and-Fire, CIF) 机制和双向上下文建模, 在中文语音识别任务上实现了识别精度与推理速度的良好平衡; 在此基础上, 石宇等人提出的 SeaCo-Paraformer^[10] 引入语义协作 (Semantic Cooperation) 解码器实现热词定制能力, 可在不重新训练主干模型的前提下显著提升特定词表 (如唤醒词、设备控制指令) 的识别率, 特别适合 IoT 设备的语音交互场景; 达摩院开源的 FunASR^[11] 集成了 Paraformer 系列模型并提供完整的训练、推理与评估流程; 王海坤等人^[1] 对语音识别技术的研究进展进行了系统梳理与展望。

在大语言模型智能体方面, 国内研究者围绕智能体的能力构建与工程应用开展了大量工作。董益宏等人^[12] 对基于大语言模型智能体的代码生成方法进行了系统综述, 归纳了智能体在任务理解、计划分解、工具调用与结果验证等环节的关键技术; 张文祺^[13] 研究了大模型推理技术及其在智能体交互决策中的应用, 表明强化大模型的推理能力能够有效提升智能体在多步交互任务中的决策可靠性; 张光华等人^[14] 提出了面向移动端智能体的端到端自动化评测框架, 将评测集管理、指标计算与结果分析组织为系统化的自动评测流程; 陈玲^[15] 设计并实现了基于大模型智能体的代码分析与评测系统, 将规则化指标计算与大模型语义分析相结合, 验证了规则评估与模型评估相结合这一技术路线的可行性。在语音识别领域, 涂冲等人^[16] 对语音识别技术及其典型应用进行了综述; 相关基础理论与工程实践可进一步参见周志华^[17]、洪青阳^[18]、韩纪庆^[19]、俞栋与邓力^[20] 等编著或翻译的著作。总体来看, 国内在工业级语音识别工具链与大语言模型智能体研究方面积累深厚、进展迅速, 智能体正朝着推理能力更强、交互决策更可靠的方向演进。

3. 选题的先进性和实用性

本系统自主设计 LLM Agent 引擎, 采用“通用工具层+领域技能层”的双层能力架构。通用工具层提供文件操作、命令执行等基础能力, 领域技能层将 ASR 优化的各处理阶段封装为符合统一接口协议的可独立调用技能 (Skill)。Agent 通过技能索引动态感知当前可用的领域能力, 根据优化状态自主编排调用策略, 而非按固定流水线顺序执行。该设计与近期 Trirat 等人提出的 AutoML-Agent^[21] (以多智能体协作完成从数据获取到模型部署的全流程自动化机器学习) 思路相通, 但本文进一步面向 ASR 的热词定制、远场噪声鲁棒性等特定问题进行领域化技能封装与优化。

将评测与分析流程系统化, 支持规则型评估器 (CER、指令召回率、Stripped_CER 等固定算法) 与 LLM 型评估器 (对 bad case 做语义归因分析) 两类评估机制。评测产出的结构化报告 (错误类型分布、混淆对、薄弱环节定位)

持久化到数据库，既供 Agent 自主读取以决定下一步优化方向，也供工程师在 Web 看板上进行跨实验对比分析。

设计完整的 Trace 链路观测能力，将 Agent 每一轮优化循环中的思考 (THINK)、执行 (EXECUTE)、反思 (REFLECT) 三阶段的全部细节结构化记录为可查询的 Span 链路数据，支持以链路图形式在 Web 控制台可视化展示，实现 Agent 决策过程的全面可追溯。

引入三方版本管理机制，对训练配置、数据集与模型 checkpoint 三者建立版本关联。每个实验节点绑定精确的版本信息，任意历史实验均可通过版本快照精确复现。平台直接面向萤石智能设备唤醒词识别的真实业务需求，部署在远程 GPU 训练集群上。平台提供 Web 管理控制台，支持团队协作与多项目管理。每位用户拥有独立的 Agent 会话实例，项目组共享实验树与数据产物，避免重复计算。版本管理使所有实验可精确复现，满足工业场景对可追溯性的要求。

4. 选题的技术难度和工作量

本课题的技术难度主要体现在以下几个方面：

Agent 决策引擎的设计与实现。需要设计动态 Prompt 构建器（根据当前实验树状态、紧凑记忆、可用技能列表等动态组装上下文）、工具/技能调度器（解析 Agent 的工具调用指令并路由到对应的 Tool 或 Skill 执行）、上下文压缩引擎（当历史对话接近 Token 上限时自动摘要压缩）、迭代预算控制器（防止无限循环消耗算力）以及轨迹记录器（将每轮决策结构化持久化）等多个核心组件的协同工作。Agent 的决策需具备可解释性——每一步决策的依据需要能够在 Trace 中被追溯。

10 个 ASR 领域技能的接口设计与封装。每个技能均须遵循统一的 Skill 接口协议，协议规定了前置条件检查、参数校验、幂等性保证以及执行结果的结构化返回等规范。同时，各技能之间存在数据依赖关系，需要合理设计执行顺序。其中，train_model 技能较为特殊，需在启动训练前检查 GPU 占用状态并向用户申请确认；evaluate_model 与 analyze_results 技能则需实现精确的 ASR 指标计算，如唤醒词首尾剥离、空参考文本排除、边界条件处理等。

多用户并发场景下的资源管理。每位用户拥有独立的 Agent 实例，但共享实验树和 GPU 资源。需要设计合理的锁机制保证实验树的一致性，以及 GPU 占用检查与排队机制避免训练冲突。

远程异构训练环境的可靠对接。远程训练环境涉及多 GPU 分布式训练 (torchrun)，需要准确捕获 FunASR 框架“返回码为 0 但无检查点产出”的静默崩溃问题，设计 SeaCo 偏置标签的启动前校验机制，以及训练日志的流式采集与解

析。

完整的 Web 平台开发。包含前端 (Vue.js 3 + Element Plus, 6 个核心页面)、后端 (FastAPI, 7 组 RESTful 接口)、数据库 (MySQL 15 张表 + Redis 状态缓存) 的全栈开发, 以及用户认证、权限管理、项目容器化隔离等工程化能力。

从工作量角度, 本课题涉及 Agent 决策引擎 (5 个核心组件)、10 个领域技能、评测与分析模块、Trace 链路观测模块、版本管理模块、Web 管理控制台 (含 Agent 交互界面、数据集管理、评测分析看板、Trace 链路视图)、RESTful API 服务层、MySQL 数据库层 (15 张核心表)、Redis 缓存层、权限与多项目隔离机制、Docker 容器化部署、远程训练环境适配层的设计与实现, 以及在 AISHELL 公开数据集和 萤石业务数据集上的端到端验证实验。

4. 参考文献

[1] 王海坤, 潘嘉, 刘聪. 语音识别技术的研究进展与展望[J]. 电信科学, 2018, 34(2): 1-11.

[2] Sculley D, Holt G, Golovin D, et al. Hidden Technical Debt in Machine Learning Systems[C]. Advances in Neural Information Processing Systems (NeurIPS), 2015: 2503-2511.

[3] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]. International Conference on Learning Representations (ICLR), 2023.

[4] Shinn N, Cassano F, Gopinath A, et al. Reflexion: Language Agents with Verbal Reinforcement Learning[C]// Advances in Neural Information Processing Systems (NeurIPS), 2023: 8534-8547.

[5] Gulati A, Qin J, Chiu C C, et al. Conformer: Convolution-augmented Transformer for Speech Recognition[C]. INTERSPEECH, 2020: 5036-5040.

[6] Park D S, Chan W, Zhang Y, et al. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition[C]. INTERSPEECH, 2019: 2613-2617.

[7] Schick T, Dwivedi-Yu J, Dessì R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools[C]// Advances in Neural Information Processing Systems (NeurIPS), 2023: 18539-18551.

[8] Chan J S, Chowdhury N, Jaffe O, et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering[C]. International

Conference on Learning Representations (ICLR), 2025.

[9] 高忠孝, 张声岫, McLoughlin I, 等. Paraformer: Fast and Accurate Parallel Transformer for Non-autoregressive End-to-End Speech Recognition[C]. INTERSPEECH, 2022: 2063-2067.

[10] 石宇, 吴志方, 高忠孝, 等. SeaCo-Paraformer: A Non-Autoregressive ASR System with Flexible and Effective Hotword Customization Ability[C]. IEEE ICASSP, 2024: 11291-11295.

[11] 高忠孝, 李哲, 王军, 等. FunASR: A Fundamental End-to-End Speech Recognition Toolkit[C]. INTERSPEECH, 2023: 1609-1613.

[12] 董益宏, 姜雪, 钱家如, 等. 基于大语言模型智能体的代码生成综述[J/OL]. 软件学报, 1-27[2026-06-23]. <https://doi.org/10.13328/j.cnki.jos.007593>.

[13] 张文祺. 大模型推理技术及其在智能体交互决策中的研究[D]. 杭州: 浙江大学, 2025.

[14] 张光华, 王宇, 张思远, 等. 面向移动端智能体的端到端自动化评测框架[J]. 软件导刊, 2026, 25(3): 9-18.

[15] 陈玲. 基于大模型智能体的代码分析与评测系统研究与应用[J]. 软件, 2025, 46(6): 43-47+174.

[16] 涂冲, 金利英, 王中任, 等. 语音识别技术及应用综述[J]. 数字技术与应用, 2025, 43(9): 179-181.

[17] 周志华. 机器学习[M]. 北京: 清华大学出版社, 2016.

[18] 洪青阳, 李琳. 语音识别: 原理与应用[M]. 北京: 电子工业出版社, 2020.

[19] 韩纪庆, 张磊, 郑铁然. 语音信号处理[M]. 北京: 清华大学出版社, 2019.

[20] 俞栋, 邓力. 解析深度学习: 语音识别实践[M]. 俞凯, 钱彦旻, 译. 北京: 电子工业出版社, 2016.

[21] Trirat P, Jeong W, Hwang S J. AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML[C]// Proceedings of the 42nd International Conference on Machine Learning (ICML). 2025: 49831-49856.

三、研究的创新之处

研究内容、拟采用的研究方法、技术路线等方面有哪些创新之处（博士必填，硕士选填）。

四、研究方法与技术路线

拟采用的设计方法，技术路线，实践方案的可行性分析。

1. 课题内容

本课题围绕“基于 Agent 架构的语音识别模型自动化优化平台”展开设计与实现，主要包含以下内容：

（1）Agent 决策引擎的设计与实现，包含 Prompt 构建器、工具注册表、技能索引、上下文管理引擎、迭代预算控制、轨迹记录器等核心组件。

（2）领域技能层的设计与实现，将 ASR 优化的 10 个处理阶段封装为符合统一接口协议的独立 Skill。

（3）评测与分析模块的设计与实现，包含规则型评估器与 LLM 型评估器，分析结果持久化存储。

（4）Trace 链路观测模块的设计与实现，将 Agent 每轮决策结构化记录为可查询的链路数据。

（5）版本管理模块的设计与实现，对配置、数据集、热词、模型建立版本关联。

（6）Web 管理控制台的设计与实现，包含 Agent 交互界面、项目管理、数据集管理、热词管理、评测分析看板、Trace 链路视图。

（7）RESTful API 服务层与 MySQL 数据库层的设计与实现。

（8）用户管理与权限控制的设计与实现（平台管理员/项目 Owner/项目 Member 三级权限）。

（9）项目容器化隔离与初始化流程的设计与实现（每个项目对应一个 Docker 容器）。

（10）远程训练环境适配层的设计与实现，包含 GPU 资源检查与训练健康监测。

（11）系统集成测试与端到端验证实验。

2. 系统需求分析（应用软件工程专业描述工具描述）

2.1 概述

本系统是一个面向 ASR 模型优化的智能管理平台。每位用户拥有独立的 Agent 会话实例，Agent 作为用户的个人优化助手，驱动“评估—诊断—决策—执行—再评估”的闭环。项目组内共享实验树与数据处理产物，避免重复计算。当涉及 GPU 密集型任务（如模型训练）时，Agent 检查资源可用性并向用户确认后方可执行。系统通过 Web 控制台为团队提供协作界面，通过数据库持久化保证实验可追溯与可复现。

2.2 功能需求

系统包含三类参与者：平台管理员、项目 Owner、项目 Member。普通用户用例图如图 4.1 所示。

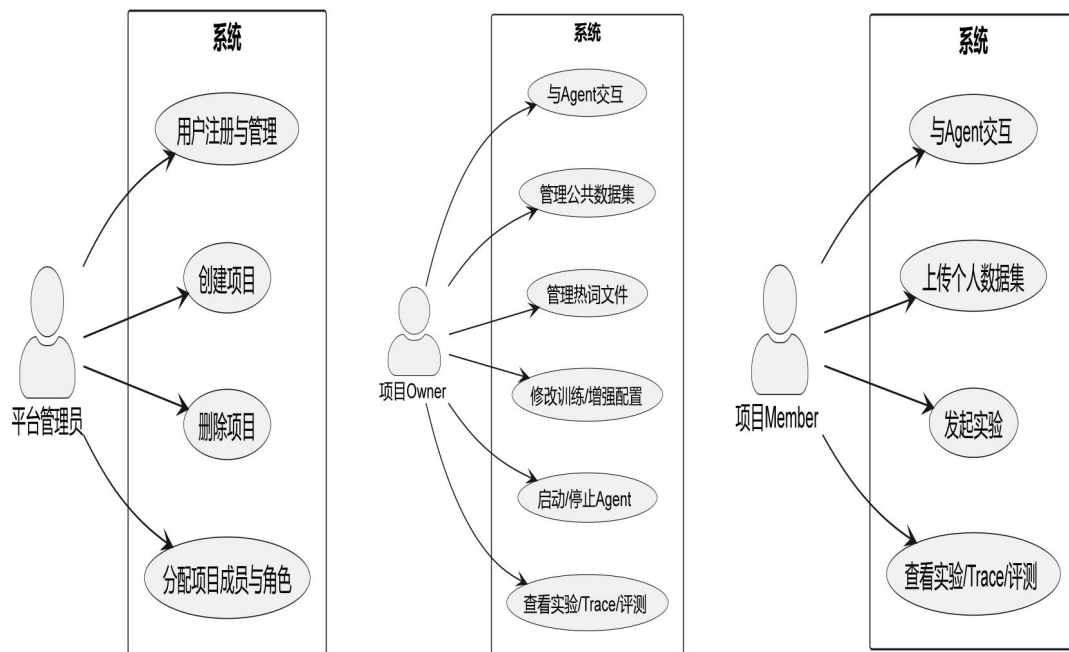


图 4.1 用户用例图

平台管理员主要负责全局维度的用户与项目管控，包括注册新用户、管理账号状态（如启用、禁用或重置密码），以及项目的全生命周期管理，如创建项目（需指定基础模型与 GPU 配置）、删除项目（涉及停止容器与归档数据），并为项目分配成员及角色。项目 Owner 拥有该项目的完整操作权限。在智能交互层面，Owner 可通过对话界面向 Agent 下达优化指令，实时查看 Agent 的思考过程，并对关键决策进行确认或否决；在资源与配置层面，Owner 负责管理公共数据集（上传至 shared 目录、删除及查看信息与版本）、维护热词文件（上传、修改与查看），并可以修改训练配

置、增强配置等全局配置文件；此外，Owner 还能控制 Agent 会话的启停，并全面查看实验树、Trace 链路及评测分析报告。项目 Member 则侧重于执行与受限查看。Member 同样可以通过对话界面发起实验请求并查看 Agent 的运行过程，也可以上传数据集至个人专属目录（datasets/{user_id}/），并查看实验树、Trace 链路、评测报告以及数据集列表。但与 Owner 不同，Member 的权限受到严格限制，既无法删除公共数据集，也不具备修改配置文件与热词文件的权限。

系统功能需求可按模块划分为智能交互模块、资源与数据管理模块、实验与分析模块、平台管控模块四个核心模块。各模块的具体功能需求如下：

（1）智能交互模块：用户通过对话界面向 Agent 下达自然语言优化指令，Agent 解析意图后自主调用领域技能执行。系统实时展示 Agent 的思考过程、当前执行的技能名称及进度状态。当涉及 GPU 训练等关键操作时，Agent 在执行前向用户展示训练配置摘要并请求确认，用户可确认或否决。Agent 会话支持异常时自动暂停并向用户报告失败原因。

（2）资源与数据管理模块：支持数据集的上传、查看、删除与版本追踪，区分共享数据集与个人数据集。热词管理功能支持热词文件的上传、格式校验、历史版本对比及单条增删。配置管理功能允许 Owner 通过 Web 界面或 Agent 交互修改训练配置及增强配置参数，每次修改自动生成新版本并记录差异。

（3）实验与分析模块：支持实验树的创建、分叉与状态管理，用户可以树形图方式查看全部实验节点的状态及关键指标。评测功能支持多维度指标计算、错误样本提取及多实验横向对比。分析功能包含错误类型分布统计、混淆对提取、薄弱环节定位及 LLM 语义归因分析。Trace 链路功能完整记录 Agent 每轮决策的思考、执行、反思三阶段，支持按时间、技能名、状态筛选及详情展开。

（4）平台管控模块：平台管理员可创建、删除项目，管理用户账号，并为项目分配成员及角色。项目创建时自动执行工作目录初始化、预训练模型链接、配置模板复制及 Docker 容器启动。权限控制基于 JWT Token 认证与 RBAC 角色模型，确保不同角色的用户只能访问其权限范围内的资源与操作。

智能交互模块作为用户与系统的唯一入口，接收用户指令后通过 Agent 调度资源与数据管理模块获取数据集与配置，再调用实验与分析模块执行训练、评测与归因分析，最终将结果反馈给用户并持久化到平台管控模块的数据库中。具体数据流为：用户下达优化指令→Agent 解析意图并查询实验树状态→读取紧凑记忆与死胡同列表→决策下一步优化方向→调用领域技能执行（数据增强/训练/评估）→结果写入数据库并更新实验树节点状态→反思并更新记忆→进入下一轮循环或结束。

以一个典型优化场景为例说明功能需求的完整流程：Owner 登录平台后进入某项目的 Agent 交互页面，输入“请针对设备 A 的远场场景优化识别效果”。Agent 接收指令后进入 THINK 阶段，读取实验树发现当前最优节点 CER 为 12.3%，查询记忆发现“速度扰动增强对远场有效”，决策采用数据增强策略。进入 EXECUTE 阶段，Agent 在实验树上分叉新节点，依次调用 `augment_data` 技能执行速度扰动增强、调用 `train_model` 技能（展示配置摘要并等待用户确认）、调用 `evaluate_model` 计算设备 A 远场场景的 CER。进入 REFLECT 阶段，Agent 发现 CER 从 12.3% 降至 10.8%，将“速度扰动+设备 A 远场有效”写入记忆，并向用户报告优化结果。整个过程的每个阶段均通过 Trace 模块完整记录，可在 Web 控制台回溯。

此外，系统还需满足以下业务约束：①同一项目内多用户共享实验树，但同一时刻只允许一个 Agent 实例占用 GPU 进行训练，其他用户的训练请求需排队等待；②增强产物为项目级共享，某用户执行增强后的产物可被同项目其他成员复用，避免重复计算；③Agent 的紧凑记忆保持恒定大小，当经验条目超过上限时自动压缩早期记录；④每个实验节点均绑定精确的配置版本、数据集版本与模型版本，保证任意历史实验可精确复现；⑤当 Agent 连续多轮 CER 无下降时，系统自动终止优化循环并生成总结报告，避免无效消耗算力。

2.3 非功能性需求

（1）可复现性：相同配置版本+数据集版本下，确定性处理阶段必须产出内容等价的产物。切分使用固定种子（`seed=42`），增强使用确定性种子。

（2）健壮性：技能执行失败时即终止后续步骤并记录失败原因；远程 SSH 连接失败须自动重连；训练过程中的静默崩溃（返回码 0 但无检查点）须被检测并报告。

（3）可观测性：Agent 每轮决策链路完整记录，各技能的执行状态（进行中/成功/失败）、耗时、输入输出摘要实时可查；训练过程中的 loss 曲线可通过 Web 看板查看。

（4）安全性：Web 控制台提供 JWT Token 认证与基于角色的访问控制（RBAC）。普通用户不能直接操作项目工作目录的关键文件（配置、热词），只能通过 Agent 间接操作或由 Owner 管理。

（5）多租户隔离：每个项目对应一个独立的 Docker 容器实例，各项目的数据集、实验记录、配置和模型相互隔离。所有项目共享 MySQL 和 Redis 实例，通过 `project_id` 字段实现逻辑隔离。

（6）性能要求：Web 页面响应时间不超过 2 秒；API 接口响应时间不超过 500ms；Agent 对话消息的首 token 延迟取决于 LLM 提供方，平台不引入额外延迟。

3. 系统概要设计（应用软件工程专业描述工具描述）

3.1 系统总体架构设计

系统采用 B/S 架构，按职责分为六层：用户层、Web 控制台层、API 服务层、数据库层、Agent 决策引擎层（含通用工具与领域技能）、远程训练环境适配层。系统总体架构如图 4.2 所示。

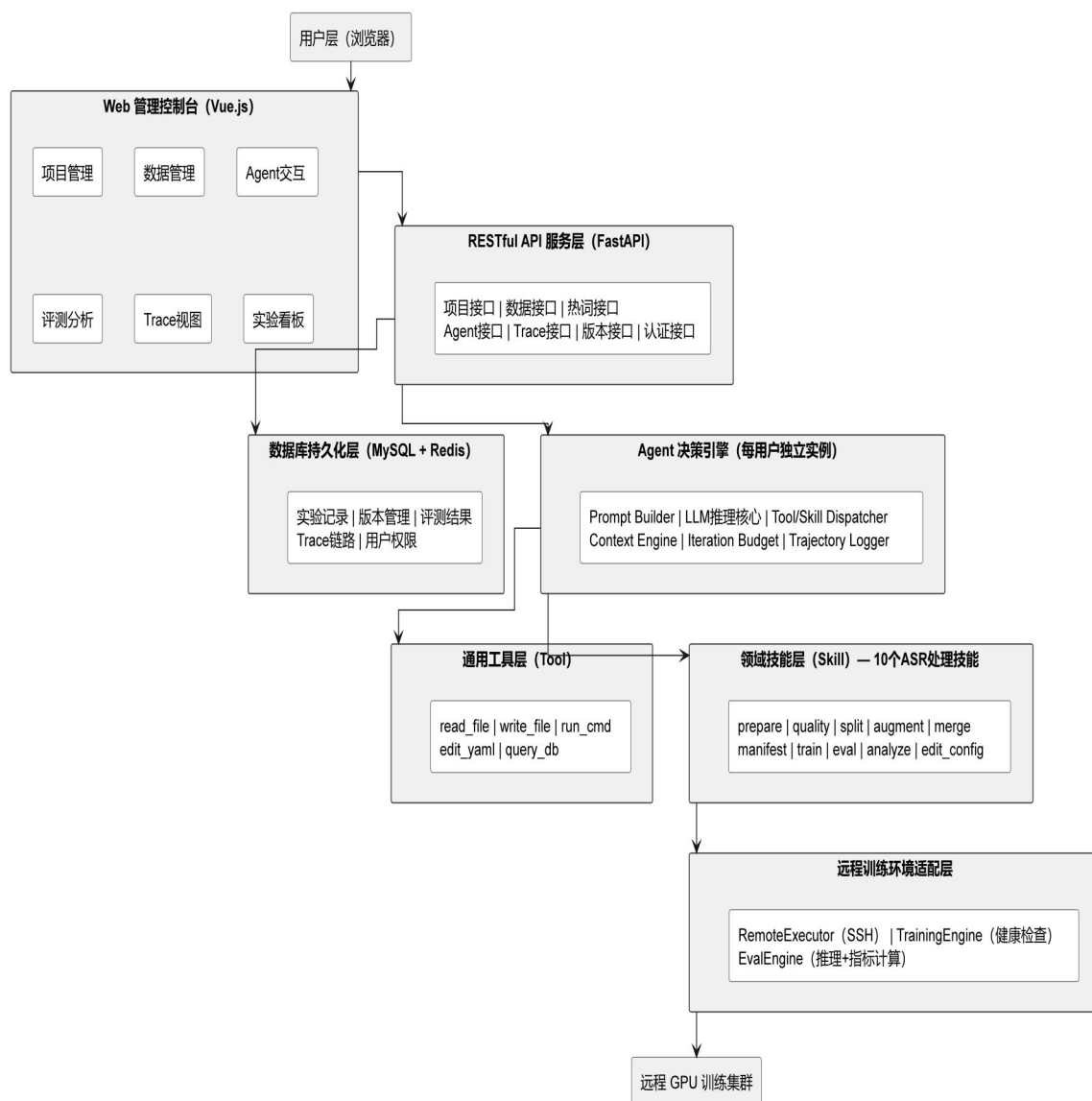


图 4.2 系统总体架构图

系统整体采用分层架构设计，自上而下协同运作。用户通过浏览器访问 Web 管理控制台，进行项目管理、Agent 交互、数据管理与结果查看等操作。Web 控制台层基于 Vue.js 与 Element Plus 构建，提供项目管理、数据管理、Agent 交互、评测分析、Trace 视图和实验看板共 6 个核心页面，负责向 API 层发送请求并展示响应结果。

API 服务层采用 FastAPI 框架，提供项目、数据、热词、Agent、Trace、版本及认证共 7 组 RESTful 接口，承担核心业务逻辑处理与数据库读写操作。数据库层由 MySQL 与 Redis 组成，MySQL 负责存储实验记录、版本信息、评测结果、Trace 链路及用户权限等所有结构化业务数据，Redis 则用于缓存 Agent 运行状态以供前端实时轮询展示；所有项目共享同一 MySQL 与 Redis 实例，并通过 project_id 字段实现数据的逻辑隔离。在核心业务逻辑侧，Agent 决策引擎层为每位用户提供独立的 Agent 会话实例，其内部集成了 Prompt Builder、LLM 推理核心、Tool/Skill Dispatcher、Context Engine、Iteration Budget 及 Trajectory Logger 等组件，并依托通用工具层提供基础操作能力，领域技能层提供 ASR 专有处理能力。底层则通过远程训练环境适配层，封装 SSH 连接管理、FunASR 训练引擎（含 GPU 检查与健康监测）及评估引擎，实现对远程 GPU 训练集群的对接。

在部署架构方面，每个项目对应一个独立的 Docker 容器实例，容器内运行 Agent 引擎与技能层代码，并挂载该项目专属的工作目录以实现环境隔离；而 Web 前端、API 服务、MySQL 与 Redis 则作为共享基础设施始终稳定运行。整体部署架构如图 4.3 所示。

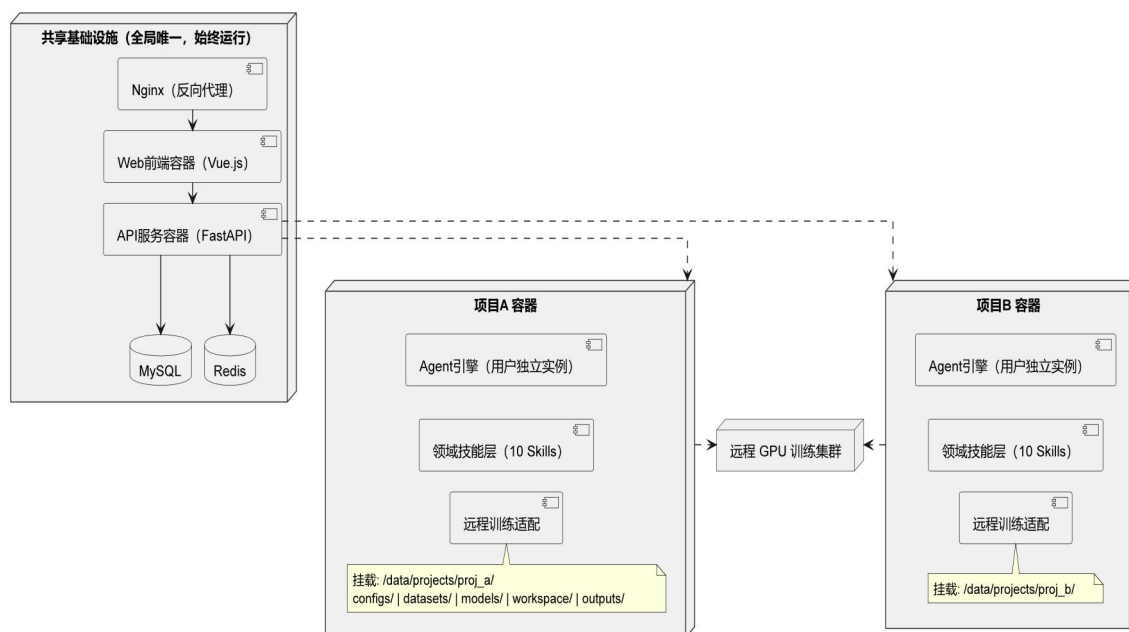


图 4.3 部署架构图

3.2 Agent 决策引擎设计

Agent 决策引擎是平台的智能核心，负责自主驱动优化循环。引擎内部由 6 个组件协同工作，其架构如图 4.4 所示。

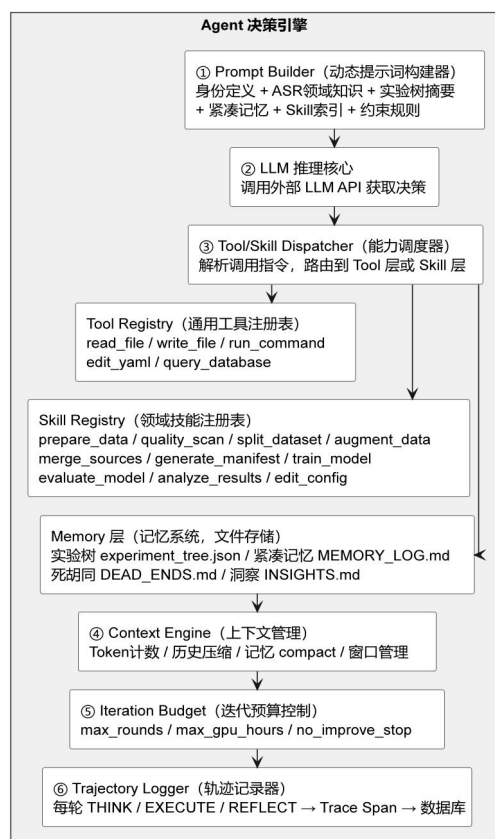


图 4.4 Agent 决策引擎内部架构图

各组件职责如下：

(1) Prompt Builder (动态提示词构建器)

负责在每轮决策前动态组装 Agent 的 system prompt。组装内容包括：身份定义（“你是 ASR 模型优化专家”）、ASR 领域知识（FunASR/SeaCo 训练经验规则）、当前实验树状态摘要、紧凑记忆（最近 N 轮经验，恒定大小）、可用技能列表及其参数规格、约束规则（“不要重复尝试死胡同中已验证无效的方向”）。Prompt Builder 确保 Agent 在每轮决策时获得充分且聚焦的上下文信息。

(2) LLM 推理核心

调用外部 LLM API（支持 GPT-4、Claude 等），将 Prompt Builder 构建的上下文与用户消息发送给 LLM，获取 Agent 的决策输出。输出包含自然语言形式的思考过程和结构化的工具调用指令（function call）。

(3) Tool/Skill Dispatcher (能力调度器)

解析 LLM 返回的工具调用指令，根据被调用的名称路由到通用工具层（Tool）或领域技能层（Skill）执行。执行完成后将结果以结构化形式返回给 LLM 继续推理。

（4） Context Engine（上下文管理引擎）

监控对话历史的 Token 数量，当接近 LLM 的上下文窗口上限时自动触发历史压缩——将早期的对话轮次摘要为简短总结，保留最近 2-3 轮的完整细节。同时负责 Agent 记忆文件（紧凑记忆、死胡同、洞察）的定期 compact 操作，确保记忆恒定大小。

（5） Iteration Budget（迭代预算控制）

防止 Agent 无限循环消耗算力。支持三种终止条件：最大轮次（max_rounds=20）、最大 GPU 训练时间（max_gpu_hours=24）、连续无改善停止（连续 N 轮 CER 无下降则终止）。预算耗尽时 Agent 自动生成优化总结报告并结束会话。

（6） Trajectory Logger（轨迹记录器）

在 Agent 每一轮决策循环中，将 THINK 阶段（Agent 的思考内容与决策依据）、EXECUTE 阶段（调用了哪些工具/技能、传递的参数、返回的结果）、REFLECT 阶段（对结果的分析与经验总结）完整记录为结构化的 Trace Span 数据，持久化到数据库的 trace 和 trace_span 表中。

Agent 的优化循环工作流程如图 4.5 所示。

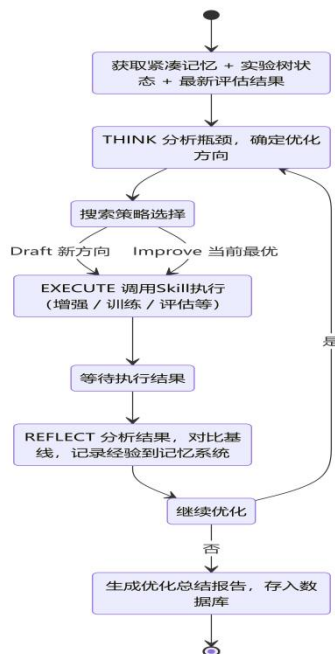


图 4.5 Agent 优化循环流程图

循环的核心逻辑为：THINK 阶段读取紧凑记忆与实验树状态，分析当前模型瓶颈，

选择搜索策略（Draft 新方向/Improve 当前最优/Debug 上次失败）；EXECUTE 阶段在实验树上分叉新节点，编辑配置并调用领域技能执行（数据增强、训练、评估等）；REFLECT 阶段解析执行结果，与基线对比，将经验记入记忆系统（有效方向记入洞察，无效方向记入死胡同），更新紧凑记忆，然后判断是否继续优化。

3.3 通用工具层与领域技能层设计

系统采用“通用工具层+领域技能层”的双层能力设计。通用工具层提供与领域无关的基础操作能力（read_file、write_file、run_command、edit_yaml、query_database），任何领域的 Agent 均可复用。领域技能层将 ASR 优化的各处理阶段封装为符合统一接口协议的独立技能，Agent 可以单独调用某个技能，也可以组合多个技能顺序执行。

每个 Skill 遵循统一的接口协议，包含以下方法与属性：name（技能名称）、description（自描述文档，注入 Agent 上下文供其了解该技能的功能与使用方式）、parameters_schema（参数 JSON Schema）、is_idempotent（是否幂等，幂等技能在产物已存在时自动跳过）、check_precondition()（前置条件检查，如 train 技能需要检查 GPU 可用性）、execute(params)（执行主逻辑，返回结构化结果）、get_output_summary()（获取输出摘要供 Agent 解读）。

本系统共设计 10 个领域技能，其功能说明如表 4.1 所示。

表 4.1 领域技能功能说明

序号	技能名	关键职责	输入	输出
1	prepare_data（数据准备）	原始数据按 FunASR 格式整理	raw 目录	wav.scp + text
2	quality_scan（质量扫描）	音频质量扫描与好坏分类	wav.scp	good/bad 清单
3	split_dataset（数据集切分）	按设备 ID 隔离切分（seed=42）	labeled 数据	train/dev/test
4	augment_data（数	训练	train 数据	增强后数据

	据增强)	集数据增强 (多策略)		
5	merge_sources (多源合并)	多数据源合并去重	多个 split 产物	merged 数据
6	generate_manifest (清单生成)	生成 FunASR 训练清单	scp+text	manifest.jsonl
7	train_model (模型训练)	GPU 检查 + 用户确认 + 分布式训练 + 健康检查	配置+清单	checkpoint
8	evaluate_model (模型评估)	模型推理 + 规则指标计算 + badcase 提取	checkpoint+测试集	metrics+badcases
9	analyze_results (结果分析)	错误类型分析 + 混淆对提取 + LLM 归因	metrics+badcases	结构化分析报告
10	edit_config (配置编辑)	修改训练 / 增强配置文件	key+value	更新后配置

其中, train_model 技能的执行流程较为特殊: 首先调用 check_precondition() 检查 GPU 占用状态, 如果 GPU 被其他用户占用则返回占用信息 (占用者、正在执行的实验、预计剩余时间) 并建议用户稍后重试; 如果 GPU 空闲, 则向用户展示训练配置摘要 (预计耗时、使用 GPU 数量、配置版本) 并请求确认; 用户确认后方可提交训练任务。

3.4 评测与分析模块设计

评测与分析以 Agent 的两个核心技能（`evaluate_model` 和 `analyze_results`）实现，产出结构化的评估报告与归因分析，结果持久化到数据库供 Web 控制台展示。评测与分析流程如图 4.6 所示。

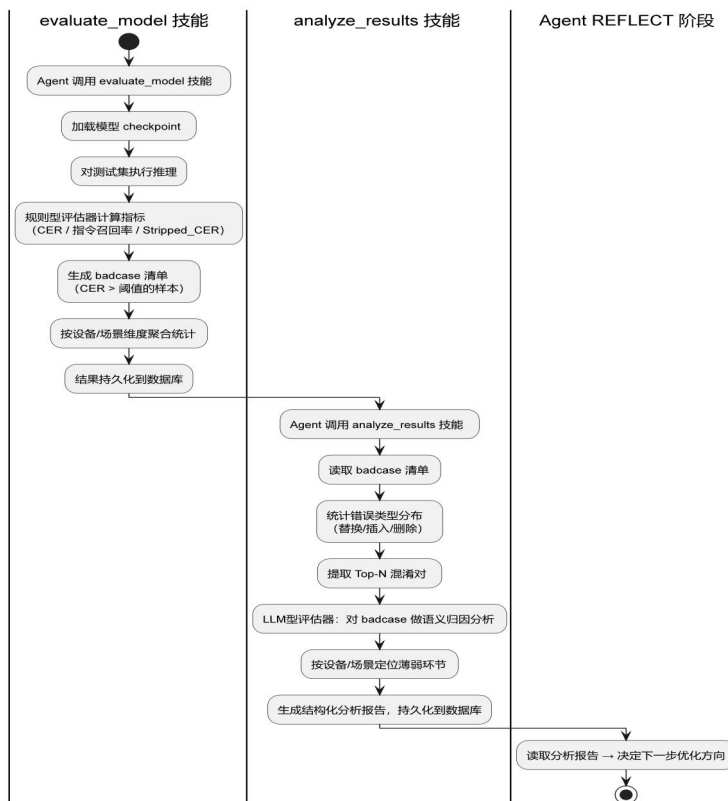


图 4.6 评测与分析流程图

`evaluate_model` 与 `analyze_results` 这两个技能紧密配合，共同构建了一套从量化评估到深度归因的完整分析链路。在模型评估阶段，系统首先加载指定的模型 checkpoint，对测试集进行批量推理以获取识别结果。随后，规则型评估器会从多个维度计算指标：不仅包括整体及按设备、场景维度的 CER（字符错误率），还会剥离唤醒词前后缀来计算控制指令的召回率（排除参考文本为空的样本），以及剥离唤醒词后的 Stripped_CER。系统会筛选出 CER 高于阈值的样本生成 badcase 清单，详细记录音频路径、参考文本、识别结果及逐字错误详情。最后，按设备和场景维度进行聚合统计，并将评测结果与 badcase 摘要持久化到数据库。进入结果分析阶段后，系统首先从数据库读取最新一次评测的 badcase 清单，统计替换、插入、删除等错误类型的分布比例，并按频次提取 Top-N 的高频混淆对（如“开灯”误识为“开等”、“小艺”误识为“小一”）。LLM 型评估器将错误分布、混淆对及设备维度指

标等结构化信息组织成提示词，调用大模型对 badcase 进行深度的语义归因分析，从而产出针对性的改进建议（例如：“设备在噪声场景下插入错误偏高，建议增加 babble 增强数据”）。在此基础上，系统进一步按设备和场景维度定位薄弱环节，生成包含 overall_cer、scene_metrics、error_distribution、top_confusions、badcase_count、llm_analysis 及 weakness_summary 等字段的结构化分析报告，并持久化至数据库。最终，Agent 在 REFLECT 阶段会提取分析报告中的 weakness_summary 和 llm_analysis 字段，作为下一轮 THINK 阶段的输入依据，从而打通了“评估→分析→决策→执行→再评估”的优化闭环。

3.5 Trace 链路观测模块设计

Trace 模块将 Agent 每轮优化循环的完整决策链路结构化记录，支持事后追溯与可视化分析。每一轮 Agent 优化对应一条 Trace 记录，内部包含多个 Span。Trace 链路示例如图 4.7 所示。

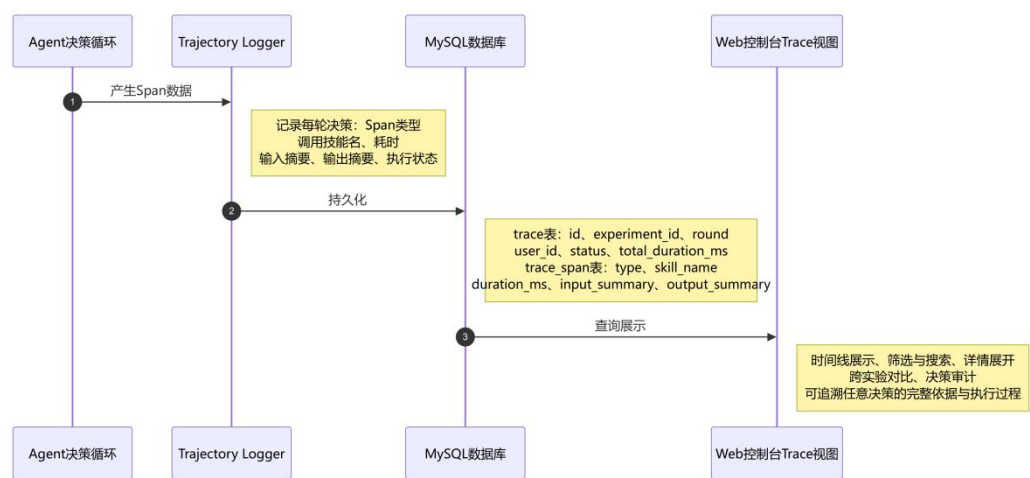


图 4.7 Trace 链路观测模块设计图

trace 表作为主表，记录了每轮优化的整体信息：每条 trace 包含唯一的 id、关联的实验 ID（experiment_id）、当前是第几轮优化（round）、由哪位用户发起（user_id）、执行状态（status）、总耗时（total_duration_ms）以及创建时间戳（created_at）。

trace_span 表则用于细化每轮优化中的具体步骤：每个 span 关联到父级 trace_id，并通过 type 字段标记其属于 THINK、EXECUTE 还是 REFLECT 阶段。对于 EXECUTE 类型的 span，会额外记录所调用的技能名称（skill_name）。此外，span 还记录了自身耗时（duration_ms）、输入输出的文本摘要

(input_summary/output_summary, 字段长度受限以避免存储膨胀) 以及该步骤的执行状态 (status)。

基于这一数据模型, Web 控制台开发了 Trace 视图页面。该页面从数据库动态查询 Trace 数据, 并以直观的时间线形式纵向展示每轮优化中各个 Span 的执行顺序与耗时情况。用户可以通过时间范围、技能名称、执行状态等多个维度灵活筛选关注的执行记录。当点击时间线上的某个 Span 节点时, 页面会展开详情面板, 完整呈现该步骤具体的输入与输出内容, 便于用户深入追溯每一步的决策逻辑与执行结果。

3.6 版本管理模块设计

版本管理对配置、数据集、热词、模型四者建立关联, 保证实验可精确复现。版本管理关系如图 4.8 所示。

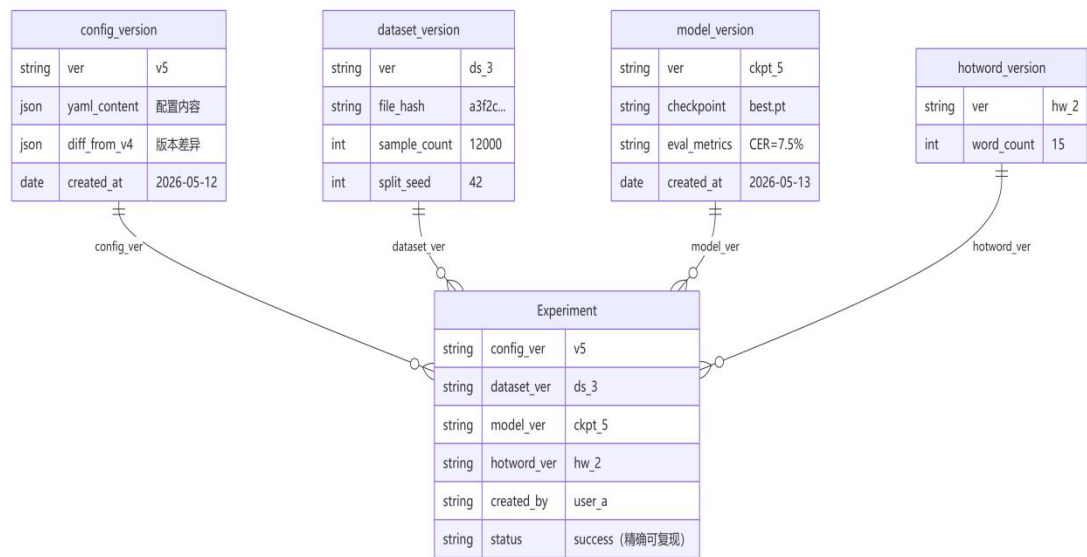


图 4.8 版本管理关联关系图

系统对每个实验节点的各项资源版本信息进行了详尽的记录。config_version 记录了训练配置 YAML 的版本, 不仅保存了完整的配置内容, 还保留了版本间的差异 (diff); dataset_version 追踪了数据集的版本, 详细记录了文件哈希值与样本数量; hotword_version 管理热词文件的版本, 并记录了所含的词条数; 而 model_version 则记录了训练产出的模型 checkpoint 版本, 并与其对应的评测指标相关联。基于这些绑定的版本信息, 任意历史实验都可以被精确复现: 只需使用同一份配置、同一版本的数据集 (在确定性种子的保证下, 数据切分将保持完全一致) 以及同一套热词配置, 即可复现出等价的训练结果。

3.7 Web 控制台与用户管理设计

Web 控制台采用 Vue.js 3 + Element Plus 开发，为不同角色的用户提供相应的功能页面。

用户管理功能：

(1) 用户注册：通过邮箱+密码注册，注册后默认为普通用户 (global_role=user)，需要平台管理员将其添加到项目才能使用项目功能。

(2) 用户登录：基于 JWT Token 认证，Token 有效期 24 小时，过期后需重新登录。

(3) 平台管理员可管理所有用户账号（启用/禁用/重置密码）。

项目管理功能（仅平台管理员）：

(4) 创建项目：指定项目名称、基础模型（如 SeaCo-Paraformer）、GPU 配置 → 系统自动按固定模板初始化工作目录 → 将预训练模型从共享模型仓库链接到项目目录 → 启动 Docker 容器 → 返回项目信息。

(5) 删除项目：停止容器、归档项目数据。

(6) 分配成员：将用户添加到项目并指定角色（Owner/Member）。

Agent 交互页面：

以对话式界面展示，左侧为对话区（用户消息与 Agent 回复交替展示），右侧为 Agent 状态面板（当前轮次、正在执行的技能、进度状态）。用户可以发送自然语言指令（如“请优先优化设备的识别效果”），Agent 理解指令后调用相应技能执行。当 train_model 技能检测到 GPU 空闲时，Agent 会在对话中展示训练配置摘要并请求用户确认。

数据集管理页面：

展示项目内所有数据集文件列表（包含文件名、上传者、大小、上传时间），Owner 可上传到 shared/ 目录，Member 可上传到个人目录。支持文件的预览（显示前几行内容）和删除（仅上传者或 Owner 可删除）。

热词管理页面（仅 Owner）：

展示当前热词列表、版本历史。支持上传新热词文件（系统自动校验格式：非空、UTF-8 编码、去重）、单独添加/删除热词、查看历史版本对比。

评测分析页面：

以图表形式展示评测结果：CER 趋势图（横轴为实验轮次，纵轴为 CER 值）、按设备维度的指标对比柱状图、错误类型分布饼图、Top-10 混淆对表格。支持选择多个实验进行横向对比。

Trace 链路页面：

以时间线形式展示 Agent 每轮优化的 Span 链路，每个 Span 显示类型（THINK/EXECUTE/REFLECT）、耗时、状态。点击 Span 展开查看输入输出详情。支持按时间范围、技能名、状态筛选。

实验看板页面：

以树形图展示实验搜索空间，节点颜色标识状态（绿色成功、红色失败、黄色进行中）。鼠标悬停节点显示关键指标（CER、创建者、时间）。支持折叠/展开子树。

3.8 权限与项目隔离设计

系统采用三级权限体系，如表 4.2 所示。

表 4.2 权限体系设计

角色	范围	核心权限
平台管理员	全局	创建/删除项目，管理用户，分配项目成员
项目 Owner	项目内	管理热词，修改全局配置，管理公共数据集，启动/停止 Agent
项目 Member	项目内	与 Agent 交互，发起实验，上传个人数据集，查看所有结果

项目内资源的共享与归属规则：

（1）实验树：项目级共享。所有成员看到同一棵实验树，每个节点记录 created_by 字段标识创建者。

（2）数据增强产物：项目级共享。某成员通过 Agent 执行增强后，产物（存放在 outputs/目录下）可被全组复用。系统自动检测已有产物避免重复执行。

（3）数据集上传：按用户隔离。Member 上传到 datasets/{user_id}/，Owner 管理 datasets/shared/。

（4）配置与热词：仅 Owner 可修改。Member 如需调整配置，需通过 Agent 交互提出建议。

项目隔离机制：每个项目对应一个独立 Docker 容器，容器挂载项目专属工作目录。所有项目共享 MySQL 和 Redis 实例（通过 project_id 逻辑隔离）。

3.10 项目初始化与容器管理

创建项目时，系统执行以下初始化流程：

1. 在数据库中创建 project 记录
2. 创建固定结构的项目工作目录：
3. 将指定的预训练基础模型从共享模型仓库软链接到项目的 models/pretrained/目录
4. 从默认模板复制训练配置与增强配置文件
5. 启动项目 Docker 容器，挂载上述工作目录
6. 将容器 ID 记录到数据库

Skill 层的所有操作均在该固定目录结构下进行，路径通过 project.yaml 配置注入。

3.11 远程训练环境适配层

远程环境适配层封装了与 GPU 训练集群的交互能力，主要包含三个组件：

RemoteExecutor（远程执行器）：管理 SSH 连接池，支持断线自动重连与会话复用。提供命令执行、文件传输、超时控制等基础能力。

TrainingEngine（训练引擎）：负责训练任务的启动与监控。关键设计包括：

（1）GPU 资源状态检查：训练前查询 GPU 占用情况，若被其他用户占用则返回占用信息

（2）配置校验：检查训练配置的关键参数（学习率、warmup 步数等）是否合理

（3）SeaCo 偏置标签启动前校验：检查热词文件中的词条是否在模型词表内

（4）torchrun 分布式训练启动与进程管理

（5）训练健康检查：检测 FunASR“返回码 0 但无检查点产出”的静默崩溃

（6）训练日志流式采集：实时解析 loss 值，推送到 Redis 供 Web 看板展示

EvalEngine（评估引擎）：负责模型推理与指标计算。加载 checkpoint 执行批量推理，计算各项评估指标。

4. 拟采用的设计方法、环境等

本系统采用面向对象的设计方法，前后端分离开发，通过 OpenAPI 3.0 规范定义 API 接口契约。领域技能层采用统一的 Skill 接口协议，便于独立开发与测试。Agent 引擎各组件通过依赖注入方式组装。开发环境及使用工具如表 4.3 所示。

表 4.3 开发环境

项目	选型	说明
后端语言	Python 3.10	Agent 引擎与技能层
Web 框架	FastAPI	API 服务层，自动生成 Swagger

		文档
前端框架	Vue.js 3 + Element Plus	Web 管理控制台
数据库	MySQL 8.0	结构化数据持久化
缓存	Redis 7	Agent 状态缓存、实时指标
ORM	SQLAlchemy 2.0	Python 数据库映射
可视化	ECharts 5	指标图表展示
ASR 框架	FunASR (SeaCo-Paraformer)	模型训练与推理
LLM 调用	OpenAI API 兼容接口	Agent 推理核心
远程执行	SSH (paramiko) + torchrun	远程训练环境对接
训练集群	远程 GPU 训练集群	GPU 计算资源
容器化	Docker + Docker Compose	项目隔离部署
测试	pytest + Hypothesis	自动化测试
接口文档	Swagger / OpenAPI 3.0	API 文档自动生成

6. 本人主要工作描述

1. 系统总体架构设计与各模块接口定义。
2. Agent 决策引擎的完整设计与实现（Prompt 构建器、能力调度器、上下文管理、迭代预算、轨迹记录器）。
3. 10 个 ASR 领域技能的接口设计与实现。
4. 评测与分析模块的实现及结果持久化。
5. Trace 链路观测模块的设计与实现。
6. 版本管理模块的设计与实现。
7. RESTful API 服务层开发（7 组接口）。
8. MySQL 数据库设计与 ORM 层开发（15 张表）。
9. Web 管理控制台前端开发（6 个核心页面，含 Agent 交互界面）。
10. 用户管理、权限控制与项目容器化隔离的实现。
11. 远程训练环境适配层开发与调试。
12. 系统集成测试与业务数据端到端验证实验。

7. 测试方案

采用“单元测试+属性测试+接口测试+端到端测试”四层策略，如表 4.4 所示。

表 4.4 测试方案

测试类型	方法	工具	覆盖范围
单元测试	示例与边界用例	pytest	Skill 执行逻辑、评估器指标计算、版本管理
属性测试	基于属性的随机验证 (≥ 100 次)	Hypothesis	数据切分确定性、合并去重正确性、幂等性
接口测试	API 联调	pytest + httpx	RESTful 接口、认证鉴权、权限控制
端到端测试	完整业务流程	pytest + mock	Agent 闭环优化全流程 (AISHELL 数据集)

8. 可行性分析

在技术可行性方面:FunASR 框架已在生产环境中广泛使用,其 SeaCo-Paraformer 模型在中文识别任务上表现优异^[10]; LLM Agent 的工具调用能力已在多个复杂任务场景中得到验证^{[3][8]}; Agent 在工具调用、推理与交互决策等方面的能力已在相关研究中得到系统梳理与验证^{[12][13]}; FastAPI + Vue.js 技术栈生态成熟,社区活跃; 远程 GPU 集群(远程 GPU 训练集群)已配置完毕并稳定运行。

在人力与条件方面:本课题的研究与实现属于所在企业 ASR 业务优化工作的核心组成部分,得到实践导师和团队的支持,具备充足的计算资源与业务数据条件。

五、预期成果

预期达到的目标和取得的成果。

1. 基于 Agent 架构的语音识别模型自动化优化平台软件系统一套,包含 Agent 决策引擎(6 个核心组件)、10 个领域技能、评测与分析模块(规则型+LLM 型评估器)、Trace 链路观测模块、版本管理模块、Web 管理控制台(6 个核心页面,含 Agent 交互界面)、RESTful API 服务层(7 组接口)、MySQL 数据库层(15 张表)、Redis 缓存层、用户管理与三级权限控制、项目 Docker 容器化隔离、远程训练环境适配层。
2. 在萤石业务数据集上经平台优化的 ASR 模型,领域 KPI(指令召回率、Stripped_CER)相较基线有显著提升。
3. 系统设计文档、API 接口文档(Swagger)、操作手册及测试报告。
4. 工程硕士学位论文 1 篇。

六、已取得的与学位申请成果研究内容相关的成果

已发表或被接收发表的学术论文或其它相关创新性成果。

七、研究计划与进度安排

应包括文献调研，工程设计，新工艺、新材料、新设备、新产品的研制和调试，实验操作，实验数据的分析处理，撰写论文等。

时间	工作内容	阶段成果
2026.06-2027.07	文献调研与技术选型；完成系统总体架构设计	文献综述、开题报告
2026.07	完成详细需求分析与概要设计；定义数据库 schema 与 API 接口规范	需求分析报告、概要设计报告
2026.07-2026.08	实现 Agent 决策引擎核心组件；实现 10 个领域技能	Agent 引擎原型、技能代码、单元测试
2026.09	实现评测与分析模块；实现 Trace 观测模块与版本管理	评测系统、Trace 模块、属性测试
2026.09-2026.10	实现 Web 管理控制台、API 服务层、数据库层、权限管理与容器管理	完整平台系统、接口测试
2026.11-2026.12	系统集成测试；开展真实业务数据多轮优化验证实验	实验数据与评估报告
2027.01-2027.02	实验数据分析，系统优化，撰写毕业论文	毕业论文初稿

2027.03	修改完善论文，准备答辩	毕业论文定稿

研究生本人签名： 刘¹⁵漠

校内导师（主导师）签名： 王鹏焜

实践导师签名： 吴长印

2026 年 7 月 10 日